

SQL - Funkcje okna (Window functions) Lab 1

Imiona i nazwiska: Marek Małek, Mateusz Lampert

Celem ćwiczenia jest przygotowanie środowiska pracy, wstępne zapoznanie się z działaniem funkcji okna (window functions) w SQL, analiza wydajności zapytań i porównanie z rozwiązaniami przy wykorzystaniu “tradycyjnych” konstrukcji SQL

Swoje odpowiedzi wpisuj w miejsca oznaczone jako:

Wyniki:

-- ...

Ważne/wymagane są komentarze.

Zamieść kod rozwiązania oraz zrzuty ekranu pokazujące wyniki, (dołącz kod rozwiązania w formie tekstowej/źródłowej)

Zwróć uwagę na formatowanie kodu

Oprogramowanie - co jest potrzebne?

Do wykonania ćwiczenia potrzebne jest następujące oprogramowanie:

- MS SQL Server - wersja 2019, 2022, 2025
- PostgreSQL - wersja 15/16/17/18
- SQLite
- Narzędzia do komunikacji z bazą danych
 - SSMS - Microsoft SQL Managment Studio
 - DDataGrip lub DBeaver
- Przykładowa baza Northwind
 - W wersji dla każdego z wymienionych serwerów

Oprogramowanie dostępne jest na przygotowanej maszynie wirtualnej

Dokumentacja/Literatura

- Kathi Kellenberger, Clayton Groom, Ed Pollack, Expert T-SQL Window Functions in SQL Server 2019, Apr 2019
- Itzik Ben-Gan, T-SQL Window Functions: For Data Analysis and Beyond, Microsoft 2020
- Kilka linków do materiałów które mogą być pomocne - <https://learn.microsoft.com/en-us/sql/t-sql/queries/select-over-clause-transact-sql?view=sql-server-ver16>

- <https://www.sqlservertutorial.net/sql-server-window-functions/>
 - <https://www.sqlshack.com/use-window-functions-sql-server/>
 - <https://www.postgresql.org/docs/current/tutorial-window.html>
 - <https://www.postgresqltutorial.com/postgresql-window-function/>
 - <https://www.sqlite.org/windowfunctions.html>
 - <https://www.sqlitetutorial.net/sqlite-window-functions/>
- W razie potrzeby - opis ikon używanych w graficznej prezentacji planu zapytania w SSMS jest tutaj:
 - <https://docs.microsoft.com/en-us/sql/relational-databases/showplan-logical-and-physical-operators-reference>

Przygotowanie

Uruchom SSMS - Skonfiguruj połączenie z bazą Northwind na lokalnym serwerze MS SQL

Uruchom DataGrip (lub Dbeaver)

- Skonfiguruj połączenia z bazą Northwind3
 - na lokalnym serwerze MS SQL
 - na lokalnym serwerze PostgreSQL
 - z lokalną bazą SQLite
-

Zadanie 1 - obserwacja

Wykonaj i porównaj wyniki następujących poleceń.

```
select avg(unitprice) avgprice
from products p;
```

```
select avg(unitprice) over () as avgprice
from products p;
```

```
select categoryid, avg(unitprice) avgprice
from products p
group by categoryid
```

```
select avg(unitprice) over (partition by categoryid) as avgprice
from products p;
```

Jaka jest są podobieństwa, jakie różnice pomiędzy grupowaniem danych a działaniem funkcji okna?

Wyniki:

```
select avg(unitprice) avgprice
from products p;
```

=> jeden wiersz, globalna średnia równa ~28.834

===

```
select avg(unitprice) over () as avgprice
from products p;
```

=> 77 wierszy, globalna średnia równa ~28.834 "doklejona" do każdego wiersza (okno zdefiniowane jako wszystkie wiersze)

===

```
select categoryid, avg(unitprice) avgprice
from products p
group by categoryid;
```

=> 8 wierszy, dla każdego categoryid mamy średnią dla danej kategorii, oryginalne wiersze zostały zredukowane do agregatów

===

```
select avg(unitprice) over (partition by categoryid) as avgprice
from products p;
```

=> 77 wierszy, do każdego wiersza doklejona jest średnia z danej kategorii (categoryid)

===

```
select p.productid, p.ProductName, p.unitprice,
       (select avg(unitprice) from products) as avgprice
from products p
where productid < 10;
```

=> 9 wierszy, do każdego doklejona jest globalna średnia (bo mamy podzapytanie, w którym nie obowiązuje filtr productid < 10) równa ~28.834

```
select p.productid, p.ProductName, p.unitprice,  
       avg(unitprice) over () as avgprice  
from products p  
where productid < 10;
```

=> 9 wierszy, do każdego doklejona jest średnia z tych 9. wierszy (filtr obowiązuje wewnątrz okna) równa ~ 31.372

Zadanie 2 - obserwacja

Wykonaj i porównaj wyniki następujących poleceń.

--1)

```
select p.productid, p.ProductName, p.unitprice,  
       (select avg(unitprice) from products) as avgprice  
from products p  
where productid < 10
```

--2)

```
select p.productid, p.ProductName, p.unitprice,  
       avg(unitprice) over () as avgprice  
from products p  
where productid < 10
```

Jaka jest różnica? Czego dotyczy warunek w każdym z przypadków?

Napisz polecenie równoważne

- 1. z wykorzystaniem funkcji okna.
 - 2. z wykorzystaniem podzapytania
-

Wyniki:

Podobnie jak w zadanie 1. - w przypadku pierwszego zapytania średnia uwzględnia wszystkie rekordy (avgprice to wynik podzapytania, w którym warunek where productid < 10 już nie obowiązuje), w przypadku drugiego zapytania warunek ten jest uwzględniany, a funkcja okna bierze średnią wyłącznie z tych odfiltrowanych 9. wierszy. W obu przypadkach wartości średniej są doklejane do wszystkich wierszy.

```
--1-with-window)  
select productid, productname, unitprice, avgprice  
from (  
    select p.productid, p.productname, p.unitprice,  
           avg(unitprice) over () as avgprice  
    from products p  
    ) sub  
where productid < 10;
```

```
--2-with-subquery)
select p.productid, p.ProductName, p.unitprice,
       (select avg(unitprice) from products where productid < 10)
from products p
where productid < 10
```

Zadanie 3

Baza: Northwind, tabela: products

Napisz polecenie, które zwraca: id produktu, nazwę produktu, cenę produktu, średnią cenę wszystkich produktów.

Napisz polecenie z wykorzystaniem z wykorzystaniem podzapytania, join'a oraz funkcji okna. Porównaj czasy oraz plany wykonania zapytań.

Przetestuj działanie w różnych SZBD (MS SQL Server, PostgreSQL, SQLite)

W SSMS włącz dwie opcje: Include Actual Execution Plan oraz Include Live Query Statistics

W DataGrip użyj opcji Explain Plan/Explain Analyze

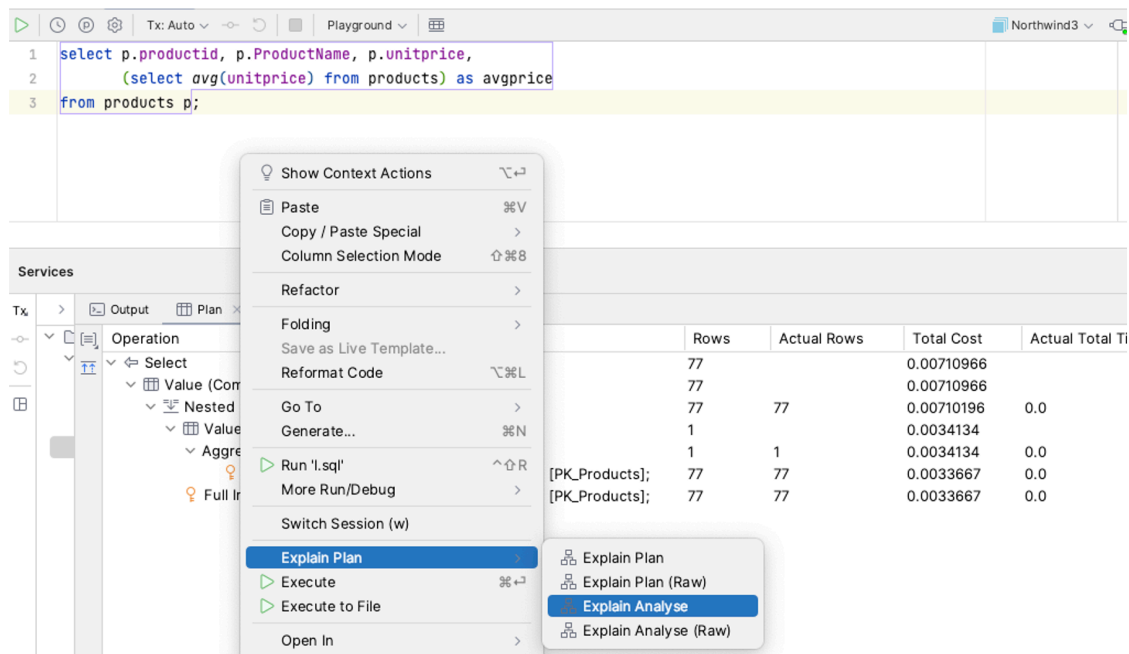


Figure 1: w:700

Wyniki:

Różnice w czasie wykonania zapytania są marginalne ze względu na rozmiar tabeli products (77 wierszy) - w rezultacie nie ma sensu porównywanie czasu wykonania zapytań.

```
--subquery
select p.productid, p.productname, p.unitprice,
       (select avg(unitprice) from products) as avgprice
from products p;
```

Postgres:

Operation	Params	Rows	Total Cost	Startup Cost
⌵ ⌵ Select				
⌵ ⌵ Full Scan (Seq Scan)	table: products;	77	3.75	1.97
⌵ ⌵ Aggregate		1	1.97	1.96
⌵ ⌵ Full Scan (Seq Scan)	table: products;	77	1.77	0.0

MSSQL:

Operation	Params	Rows	Total Cost
⌵ ⌵ Select		77	0.00859114
⌵ ⌵ Value (Compute Scalar)		77	0.00859114
⌵ ⌵ Nested Loops (Inner Join - Nest		77	0.00858344
⌵ ⌵ Value (Compute Scalar)		1	0.00415414
⌵ ⌵ Aggregate (Aggregate - Stre		1	0.00415414
⌵ ⌵ Full Index Scan (Cluste	table: [dbo].[products]; index: [pk_products];	77	0.00410744
⌵ ⌵ Full Index Scan (Clustered In	table: [dbo].[products]; index: [pk_products];	77	0.00410744

SQLite:

Operation	Params	Raw Desc
⌵ ⌵ Select		
⌵ ⌵ Full Scan	table: p;	SCAN p
⌵ ⌵ Subquery	Scalar;	SCALAR SUBQUERY 1
⌵ ⌵ Full Scan	table: products;	SCAN products

```
--join
select p.productid, p.productname, p.unitprice, sub.avgprice
from products p
cross join (
    select avg(unitprice) as avgprice
    from products
) sub;
```

Postgres:

Operation	Params	Rows	Total Cost	Startup Cost
↳ Select				
↳ Nested Loops (Nested Loop)		77	4.51	1.96
↳ Aggregate		1	1.97	1.96
↳ Full Scan (Seq Scan)	table: products;	77	1.77	0.0
↳ Full Scan (Seq Scan)	table: products;	77	1.77	0.0

MSSQL:

Operation	Params	Rows	Total Cost
↳ Select		77	0.00858344
↳ Nested Loops (Inner Join)		77	0.00858344
↳ Value (Compute Scalar)		1	0.00415414
↳ Aggregate (Aggregate)		1	0.00415414
↳ Full Index Scan (table: [dbo].[products]; index: [pk_products];)		77	0.00410744
↳ Full Index Scan (Clustered index: [pk_products]; table: [dbo].[products];)		77	0.00410744

SQLite:

Operation	Params	Raw Desc
↳ Select		
↳ Operation (MATERIALIZED)		MATERIALIZED sub
↳ Full Scan	table: products;	SCAN products
↳ Full Scan	table: p;	SCAN p
↳ Full Scan	table: sub;	SCAN sub

```
--window function
select p.productid, p.productname, p.unitprice,
       avg(p.unitprice) over () as avgprice
from products p;
```

Postgres:

Operation	Params	Rows	Total Cost	Startup Cost
↳ Select				
↳ Transformation (WindowAgg)		77	2.73	2.7
↳ Full Scan (Seq Scan)	table: products;	77	1.77	0.0

MSSQL:

Operation	Params	Rows	Total Cost
↳ Select		77	0.0055688
↳ Nested Loops (Inner Join - Nested Loops)		77	0.0055688
↳ Temporary (Lazy Spool - Table Spool)		1	0.00439971
↳ Transformation (Segment)		77	0.00425358
↳ Full Index Scan (Clustered Index Scan)	table: [dbo].[products]; index: [pk_products];	77	0.00410744
↳ Nested Loops (Inner Join - Nested Loops)		77	2.92272E-5
↳ Value (Compute Scalar)		1	1.6075E-5
↳ Aggregate (Aggregate - Stream Aggregate)		1	1.46136E-5
↳ Temporary (Lazy Spool - Table Spool)		77	0.0
↳ Temporary (Lazy Spool - Table Spool)		77	0.0

SQLite:

Operation	Params	Raw Desc
↳ Select		
↳ Operation (CO-ROUTINE)		CO-ROUTINE (subquery-2)
↳ Full Scan	table: p;	SCAN p
↳ Full Scan	table: (subquery-2);	SCAN (subquery-2)

Zadanie 4

Baza: Northwind, tabela products

Napisz polecenie, które zwraca: id produktu, nazwę produktu, cenę produktu, średnią cenę produktów w kategorii, do której należy dany produkt. Wyświetl tylko pozycje (produkty) których cena jest większa niż średnia cena.

Napisz polecenie z wykorzystaniem podzapytania, join'a oraz funkcji okna. Porównaj zapytania. Porównaj czasy oraz plany wykonania zapytań.

Przetestuj działanie w różnych SZBD (MS SQL Server, PostgreSQL, SQLite)

Wyniki: W MSSQL plan wykonania jest dostosowany przez optymalizator i podmieniony na najbardziej optymalny (totalTime 0.0). W przypadku Postgresql i SQLite plan wykonania jest zgodny z napisanym zapytaniem.

```
--subquery
select p1.ProductID, p1.ProductName, p1.UnitPrice from Products p1
where p1.UnitPrice > (select avg(p2.UnitPrice) from Products p2 where
p1.CategoryID = p2.CategoryID)
order by p1.ProductID
```

Postgres:

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
↳ Select								Planning Time = 0.073...
↳ Sort		26	27	156.64	0.421	156.57	0.42	Parallel Aware = false;A...
↳ Full Scan (Seq Scan)	table: products;	26	27	155.96	0.414	0.0	0.045	Parent Relationship = O...
↳ Aggregate		1	1	2.0	0.005	1.99	0.005	Strategy = PlainPartial ...
↳ Full Scan (Seq Scan)	table: products;	10	11	1.96	0.004	0.0	0.001	Parent Relationship = O...

MSSQL:

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Raw Desc
↳ Select		77		0.0298979		CardinalityEstimatio...
↳ Sort		77	27	0.0298979	0.0	AvgRowSize = 63;Es...
↳ Nested Loops (Inner Join - Nested Loops)		77	27	0.0177838	0.0	AvgRowSize = 63;Es...
↳ Temporary (Lazy Spool - Table Spool)		8	9	0.0165636	0.0	AvgRowSize = 67;Es...
↳ Transformation (Segment)		77	77	0.016411	0.0	AvgRowSize = 67;Es...
↳ Sort		77	77	0.0162585	0.0	AvgRowSize = 67;Es...
↳ Full Index Scan (Clustered Index Scan)	table: [dbo].[products]; index: [pk_products];	77	77	0.00410744	0.0	AvgRowSize = 67;Es...
↳ Nested Loops (Inner Join - Nested Loops)		9.625	27	3.05068E-5	0.0	AvgRowSize = 67;Es...
↳ Value (Compute Scalar)		1		1.67787E-5		AvgRowSize = 67;Es...
↳ Aggregate (Aggregate - Stream Aggregate)		1	8	1.52534E-5	0.0	AvgRowSize = 67;Es...
↳ Temporary (Lazy Spool - Table Spool)		9.625	77	0.0	0.0	AvgRowSize = 67;Es...
↳ Temporary (Lazy Spool - Table Spool)		9.625	77	0.0	0.0	AvgRowSize = 67;Es...

SQLite:

Operation	Params	Raw Desc
↳ Select		
↳ Full Scan	table: p1;	SCAN p1
↳ Subquery	Scalar; Correlated;	CORRELATED SCALAR SUBQUERY 1
↳ Index Scan	table: p2; index: CategoryID;	SEARCH p2 USING INDEX CategoryID (CategoryID=?)

```
--join
with t as (
    select CategoryID, avg(UnitPrice) as AvgPrice from Products
    group by CategoryID
)
select p.ProductID, p.ProductName, p.UnitPrice, t.AvgPrice from Products p
join t on p.CategoryID = t.CategoryID
where p.UnitPrice > t.AvgPrice
order by p.ProductID;
```

Postgres:

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
✚ Select								Planning Time = 0.1...
✚ Sort		26	27	5.12	0.059	5.06	0.057	Parallel Aware = fals...
✚ Hash Join		26	27	4.45	0.052	2.44	0.039	Parent Relationship...
Full Scan (Seq Scan)	table: products;	77	77	1.77	0.011	0.0	0.008	Parent Relationship...
Transformation (Hash)		8	8	2.33	0.026	2.33	0.025	Parent Relationship...
Access (Subquery Scan)		8	8	2.33	0.024	2.15	0.022	Parent Relationship...
Aggregate		8	8	2.25	0.023	2.15	0.022	Strategy = Hashed;...
Full Scan (Seq Scan)	table: products;	77	77	1.77	0.005	0.0	0.001	Parent Relationship...

MSSQL:

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Raw Desc
✚ Select		77		0.0298979		CardinalityEstimati...
✚ Sort		77	27	0.0298979	0.0	AvgRowSize = 71,E...
Nested Loops (Inner Join - Nested Loops)		77	27	0.0177838	0.0	AvgRowSize = 71,E...
Temporary (Lazy Spool - Table Spool)		8	9	0.0165636	0.0	AvgRowSize = 67,E...
Transformation (Segment)		77	77	0.016411	0.0	AvgRowSize = 67,E...
Sort		77	77	0.0162585	0.0	AvgRowSize = 67,E...
Full Index Scan (Clustered Index Scan)	table: [dbo].[products]; index: [pk_products];	77	77	0.00410744	0.0	AvgRowSize = 67,E...
Nested Loops (Inner Join - Nested Loops)		9.625	27	3.05068E-5	0.0	AvgRowSize = 67,E...
Value (Compute Scalar)		1		1.67787E-5		AvgRowSize = 67,E...
Aggregate (Aggregate - Stream Aggregate)		1	8	1.52534E-5	0.0	AvgRowSize = 67,E...
Temporary (Lazy Spool - Table Spool)		9.625	77	0.0	0.0	AvgRowSize = 67,E...
Temporary (Lazy Spool - Table Spool)		9.625	77	0.0	0.0	AvgRowSize = 67,E...

SQLite:

Operation	Params	Raw Desc
✚ Select		CO-ROUTINE t
Operation (CO-ROUTINE)		SCAN Products USING INDEX CategoryID
Index Scan	table: Products; index: CategoryID;	SCAN t
Full Scan	table: t;	SEARCH p USING INDEX CategoryID (CategoryID=?)
Index Scan	table: p; index: CategoryID;	USE TEMP B-TREE FOR ORDER BY
Order By		

```
--window function
with t as (
    select p.ProductID, p.ProductName, p.UnitPrice, avg(p.UnitPrice)
over(partition by p.CategoryID) as AvgPrice from Products p
)
select ProductID, ProductName, UnitPrice, AvgPrice from t
where UnitPrice > AvgPrice
order by ProductID;
```

Postgres:

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
↕ Select								Planning Time = 0.07;Trigg...
↳ Sort		26	27	156.64	0.357	156.57	0.356	Parallel Aware = false;Asyn...
↳ Full Scan (Seq Scan)	table: products;	26	27	155.96	0.351	0.0	0.051	Parent Relationship = Oute...
↳ Aggregate		1	1	2.0	0.004	1.99	0.004	Strategy = PlainPartial Mo...
↳ Full Scan (Seq Sc	table: products;	10	11	1.96	0.003	0.0	0.001	Parent Relationship = Oute...

MSSQL:

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Raw Desc
↕ Select		23.1	27	0.0293084	0.0	CardinalityEstimationModelVersion = 160;Query...
↳ Sort		23.1	27	0.0293084	0.0	AvgRowSize = 71;EstimateCPU = 0.000263308;...
↳ Filter		23.1	27	0.0177838	0.0	AvgRowSize = 71;EstimateCPU = 3.696e-05;Est...
↳ Nested Loops (Inner Join - Nested Loops)		77	77	0.0177469	0.0	AvgRowSize = 71;EstimateCPU = 0.00117451;Es...
↳ Temporary (Lazy Spool - Table Spool)		8	9	0.0165266	0.0	AvgRowSize = 67;EstimateCPU = 0;EstimateIO ...
↳ Transformation (Segment)		77	77	0.0163741	0.0	AvgRowSize = 67;EstimateCPU = 0.000152534;...
↳ Sort		77	77	0.0162215	0.0	AvgRowSize = 67;EstimateCPU = 0.000852833...
↳ Full Index Scan (Clustered Index Scan)	table: [dbo].[products]; index: [pk_products];	77	77	0.00410744	0.0	AvgRowSize = 67;EstimateCPU = 0.0002417;Es...
↳ Nested Loops (Inner Join - Nested Loops)		9.625	77	3.05088E-5	0.0	AvgRowSize = 67;EstimateCPU = 1.52534e-05;...
↳ Value (Compute Scalar)		1		1.67787E-5		AvgRowSize = 67;EstimateCPU = 1.52534e-06;...
↳ Aggregate (Aggregate - Stream Aggregate)		1	8	1.52534E-5	0.0	AvgRowSize = 67;EstimateCPU = 1.52534e-05;...
↳ Temporary (Lazy Spool - Table Spool)		9.625	77	0.0	0.0	AvgRowSize = 67;EstimateCPU = 0;EstimateIO ...
↳ Temporary (Lazy Spool - Table Spool)		9.625	77	0.0	0.0	AvgRowSize = 67;EstimateCPU = 0;EstimateIO ...

SQLite:

Operation	Params	Raw Desc
↕ Select		CO-ROUTINE t
↳ Operation (CO-ROUTINE)		CO-ROUTINE (subquery-3)
↳ Index Scan	table: p; index: CategoryID;	SCAN p USING INDEX CategoryID
↳ Full Scan	table: (subquery-3);	SCAN (subquery-3)
↳ Full Scan	table: t;	SCAN t
Order By		USE TEMP B-TREE FOR ORDER BY

Zadanie 5

Oryginalna baza Northwind jest bardzo mała. Warto zaobserwować działanie na nieco większym zbiorze danych.

Baza Northwind3 zawiera dodatkową tabelę product_history

- 2,2 mln wierszy

Bazę Northwind3 można pobrać z moodle (zakładka - Backupy baz danych)

Można też wygenerować tabelę product_history przy pomocy skryptu

Skrypt dla SQL Sserver

Stwórz tabelę o następującej strukturze:

```

create table product_history(
    id int identity(1,1) not null,
    productid int,
    productname varchar(40) not null,
    supplierid int null,
    categoryid int null,
    quantityperunit varchar(20) null,
    unitprice decimal(10,2) null,
    quantity int,
    value decimal(10,2),
    date date,
    constraint pk_product_history primary key clustered
        (id asc )
)

```

Wygeneruj przykładowe dane:

Dla 30000 iteracji, tabela będzie zawierała nieco ponad 2mln wierszy (dostostu ograniczenie do możliwości swojego komputera)

Skrypt dla SQL Sserver

```

declare @i int
set @i = 1
while @i <= 30000
begin
    insert product_history
    select productid, ProductName, SupplierID, CategoryID,
        QuantityPerUnit, round(RAND()*unitprice + 10,2),
        cast(RAND() * productid + 10 as int), 0,
        dateadd(day, @i, '1940-01-01')
    from products
    set @i = @i + 1;
end;

update product_history
set value = unitprice * quantity
where 1=1;

```

Skrypt dla Postgresql

```
create table product_history(  
  id int generated always as identity not null  
    constraint pkproduct_history  
      primary key,  
  productid int,  
  productname varchar(40) not null,  
  supplierid int null,  
  categoryid int null,  
  quantityperunit varchar(20) null,  
  unitprice decimal(10,2) null,  
  quantity int,  
  value decimal(10,2),  
  date date  
);
```

Wygeneruj przykładowe dane:

Skrypt dla Postgresql

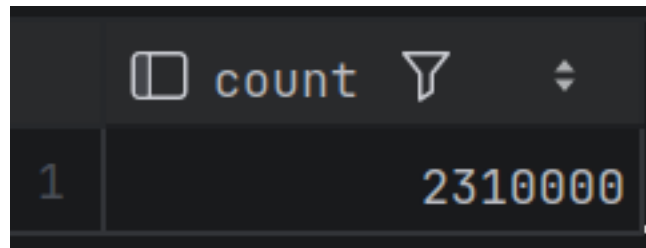
```
do $$  
begin  
  for cnt in 1..30000 loop  
    insert into product_history(productid, productname, supplierid,  
      categoryid, quantityperunit,  
      unitprice, quantity, value, date)  
    select productid, productname, supplierid, categoryid,  
      quantityperunit,  
      round((random()*unitprice + 10)::numeric,2),  
      cast(random() * productid + 10 as int), 0,  
      cast('1940-01-01' as date) + cnt  
    from products;  
  end loop;  
end; $$;  
  
update product_history  
set value = unitprice * quantity  
where 1=1;
```

Wykonaj polecenia: select count(*) from product_history, potwierdzające wykonanie zadania

Wyniki:

```
select count(*) from product_history;
```

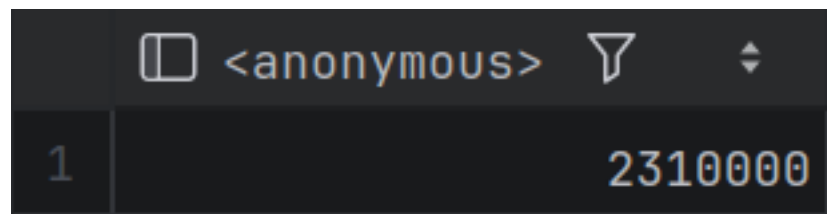
Postgres:



	count
1	2310000

Figure 2: alt text

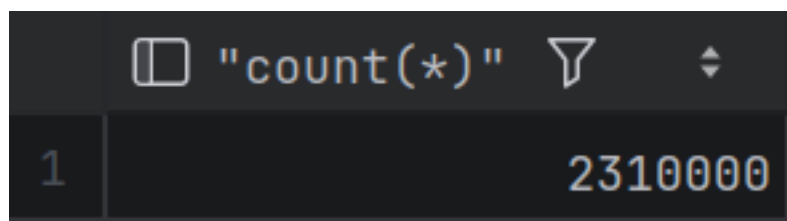
MSSQL:



	<anonymous>
1	2310000

Figure 3: alt text

SQLite:



	"count(*)"
1	2310000

Figure 4: alt text

Zadanie 6

Baza: Northwind, tabela product_history

Napisz polecenie, które zwraca: id pozycji, id produktu, nazwę produktu, id_kategorii, cenę produktu, średnią cenę produktów w kategorii do której należy dany produkt. Wyświetl tylko pozycje (produkty) których cena jest większa niż średnia cena.

W przypadku długiego czasu wykonania ogranicz zbiór wynikowy do kilkuset/kilku tysięcy wierszy pomocna może być konstrukcja with

```
with t as (  
  
....  
)  
select * from t  
where id between ....
```

Napisz polecenie z wykorzystaniem podzapytania, join'a oraz funkcji okna. Porównaj zapytania. Porównaj czasy oraz plany wykonania zapytań.

Przetestuj działanie w różnych SZBD (MS SQL Server, PostgreSQL, SQLite)

Wyniki: Z podzapytaniem tylko MSSQL poradził sobie w skończonym czasie (457ms, 45ms dla ograniczonego zbioru), prawdopodobnie przez paralelizm. PostgreSQL i SQLite wypadły znacznie gorzej ~1 minuta na ograniczonym zbiorze, a dla pełnego zbioru zapytanie nie zostało ukończzone w rozsądnym czasie. Przy joinie wyniki wyniosły odpowiednio MSSQL 458ms, PostgreSQL 1s, SQLite 1s. Przy funkcji okna MSSQL znów był najszybszy ~500ms, PostgreSQL 1.6s, SQLite 2s.

DataGrip wskazał, że w przypadku MSSQL warto dodać indeks na kolumnie categoryid w tabeli product_history.

--subquery

```
select p1.productid, p1.productname, p1.unitprice from product_history p1
where p1.unitprice > (select avg(p2.unitprice) from product_history p2 where
p1.categoryid = p2.categoryid )
order by p1.productid;
```

MSSQL:

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Raw Desc
⌵ ⚙ Select		2.31E+6		57.4659		BatchModeOnRowSto...
⌵ Transformation (Gather Streams - Parallelism)		2.31E+6	775336	57.4659	78.0	AvgRowSize = 44;Esti...
⌵ Sort		2.31E+6	775336	47.2966	127.0	AvgRowSize = 44;Esti...
⌵ Hash Join (Inner Join - Hash Match)		2.31E+6	775336	44.4944	8.0	AvgRowSize = 44;Esti...
⌵ Value (Compute Scalar)		8	8	22.2035	0.0	AvgRowSize = 28;Esti...
⌵ Aggregate (Aggregate - Hash Match)		8	8	22.2035	5.0	AvgRowSize = 28;Esti...
⌵ Full Index Scan (Clustered Index Scan)	table: [dbo].[product_history]; index: [pk_product_history];	2.31E+6	2310000	21.98	17.0	AvgRowSize = 20;Esti...
⌵ Full Index Scan (Clustered Index Scan)	table: [dbo].[product_history]; index: [pk_product_history];	2.31E+6	2310000	21.98	29.0	AvgRowSize = 48;Esti...

-- subquery (reduced)

```
with t as (
select p1.id, p1.productid, p1.productname, p1.unitprice from product_history p1
where p1.unitprice > (select avg(p2.unitprice) from product_history p2 where
p1.categoryid = p2.categoryid )
)
select * from t
where t.id between 1000 and 1500
order by t.productid
```

Postgres:

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
⌵ ⚙ Select								Planning Time = ...
⌵ Sort		177	165	4.686949224E7	68788.399	4.68694918E7	68788.392	Parallel Aware = ...
⌵ Index Scan	table: product_history; index: pkproduct_history;	177	165	4.686948519E7	68788.189	0.43	899.817	Parent Relations...
⌵ Aggregate		1	1	88432.89	137.169	88432.88	137.169	Strategy = Plain;...
⌵ Full Scan (Seq table: product_history;		288750	316287	87711.0	122.301	0.0	2.383	Parent Relations...

SQLite:

Operation	Params	Raw Desc
⌵ ⚙ Select		
⌵ Index Scan	table: p1;	SEARCH p1 USING INTEGER PRIMARY KEY (rowid=? AND rowid<?)
⌵ Subquery	Scalar, Correlated;	CORRELATED SCALAR SUBQUERY 1
⌵ Full Scan	table: p2;	SCAN p2
Order By		USE TEMP B-TREE FOR ORDER BY


```
--join
with t as (
    select categoryid, avg(unitprice) as avgprice from product_history
    group by categoryid
)
select p.productid, p.productname, p.unitprice, t.AvgPrice from product_history p
join t on p.categoryid = t.categoryid
where p.unitprice > t.AvgPrice
order by p.productid;
```

Postgres:

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
✎ Select								Planning Time = 0...
✎ Unknown (Gather Merge)		641666	773614	282293.57	676.807	207427.31	606.163	Parallel Aware = fal...
✎ Sort		320833	257871	207229.37	610.552	206427.28	596.305	Parent Relationshi...
✎ Hash Join		320833	257871	165020.26	558.933	93486.28	417.849	Parent Relationshi...
Full Scan (Seq Scan)	table: product_history;	962500	770000	68461.0	47.579	0.0	0.012	Parent Relationshi...
Transformation (Hash)		8	8	93486.18	417.357	93486.18	417.355	Parent Relationshi...
Access (Subquery Sc		8	8	93486.18	417.351	93486.0	417.348	Parent Relationshi...
Aggregate		8	8	93486.1	417.344	93486.0	417.341	Strategy = Hashed...
Full Scan (Seq	table: product_history;	2310000	2310000	81936.0	167.11	0.0	0.004	Parent Relationshi...

MSSQL:

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Raw Desc
✎ Select		2.31E+6		57.9443		BatchModeOnRowSto...
Transformation (Gather Streams - Parallelism)		2.31E+6	775336	57.9443	80.0	AvgRowSize = 61;Esti...
Sort		2.31E+6	775336	47.2966	152.0	AvgRowSize = 61;Esti...
Hash Join (Inner Join - Hash Match)		2.31E+6	775336	44.4944	10.0	AvgRowSize = 61;Esti...
Value (Compute Scalar)		8	8	22.2035	0.0	AvgRowSize = 28;Esti...
Aggregate (Aggregate - Hash Match)		8	8	22.2035	6.0	AvgRowSize = 28;Esti...
Full Index Scan (Clustered Index Scan)	table: [dbo].[product_history]; index: [pk_product_history];	2.31E+6	2310000	21.98	17.0	AvgRowSize = 20;Esti...
Full Index Scan (Clustered Index Scan)	table: [dbo].[product_history]; index: [pk_product_history];	2.31E+6	2310000	21.98	28.0	AvgRowSize = 48;Esti...

SQLite:

Operation	Params	Raw Desc
✎ Select		CO-ROUTINE t
Operation (CO-ROUTINE)		SCAN product_history
Full Scan	table: product_history;	SCAN product_history
Group By		USE TEMP B-TREE FOR GROUP BY
Full Scan	table: p;	SCAN p
Unknown		BLOOM FILTER ON t (categoryid=?)
Index Scan	table: t; index: (categoryid=?);	SEARCH t USING AUTOMATIC COVERING INDEX (categoryid=?)
Order By		USE TEMP B-TREE FOR ORDER BY

```
--window function
with t as (
    select p.productid, p.productname, p.unitprice, avg(p.unitprice)
over(partition by p.categoryid) as avgprice from product_history p
)
select productid, productname, unitprice, avgprice from t
where unitprice > avgprice
order by productid;
```

Postgres:

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
↕ Select								Planning Time = 0...
↳ Sort		770000	773614	641054.6	1551.254	639129.6	1516.452	Parallel Aware = fal...
↳ Access (Subquery Scan)		770000	773614	505940.78	1365.618	436640.78	697.522	Parent Relationship...
↳ Transformation (Window/		2310000	2310000	477065.78	1260.429	436640.78	697.515	Parent Relationship...
↳ Sort		2310000	2310000	442415.78	740.655	436640.78	622.099	Parent Relationship...
↳ Full Scan (Seq	table: product_history;	2310000	2310000	81936.0	208.326	0.0	40.552	Parent Relationship...

MSSQL:

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Raw Desc
↕ Select		693000		28.9977		BatchModeOn...
↳ Transformation (Gather Streams - Parallelism)		693000	775336	28.9977	109.0	
↳ Sort		693000	775336	25.7835	158.0	AvgRowSize = ...
↳ Filter		693000	775336	25.0115	1.0	AvgRowSize = ...
↳ Value (Compute Scalar)		2.31E+6	2310000	24.8729	17.0	AvgRowSize = ...
↳ Unknown (Window Aggregate)		2.31E+6	2310000	24.8729	9.0	AvgRowSize = ...
↳ Sort		2.31E+6	2310000	24.7823	31.0	AvgRowSize = ...
↳ Full Index Scan (Clustered Index Scan)	table: [dbo].[product_history]; index: [pk_product_history];	2.31E+6	2310000	21.98	32.0	AvgRowSize = ...

SQLite:

Operation	Params	Raw Desc
↕ Select		CO-ROUTINE t
↳ Operation (CO-ROUTINE)		CO-ROUTINE (subquery-3)
↳ Operation (CO-ROUTINE)		SCAN p
↳ Full Scan	table: p;	USE TEMP B-TREE FOR ORDER BY
Order By		SCAN (subquery-3)
↳ Full Scan	table: (subquery-3);	SCAN t
↳ Full Scan	table: t;	USE TEMP B-TREE FOR ORDER BY
Order By		

zadanie pkt

1 1

2 1

3 1

4 1

5 1

6 2

razem: 7